

Member ID: _____

Time: _____

Rank: _____



C# PROGRAMMING

(330)

STATE 2023

PRODUCTION:

StateDataReader

_____ (435 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for grading.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder. The project folder for you has already been provided: StateDataReader
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.
5. You will need to use Visual Studio 2022 Community Edition to complete this exam

*Note that the flash drive letter may *not* be the same when the program is graded as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter. It is HIGHLY recommended that you place all of the files into one folder.

The graders will *not* compile or alter your source code to correct for this.
Submissions that do *not* contain source code will *not* be graded.

Assumptions to make when taking this assessment:

- The form design is already created and does not need to be adjusted.
- There is input required by the user.
- The program will only exit when the VS stop button is manually clicked.
- You CANNOT create any other classes (internal or external).
- All objects (buttons, list boxes, labels, panels, etc.) needed for the design aspect of the form have been provided.
- The needed text files are located in the project folder at the following location:
 - \StateDataReader\StateDataReader\bin\Debug
 - There are 4 text files you will need (provided). Nothing in this folder needs to be adjusted
- All required packages needed to complete have been provided for you

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines (if any), functions (if any), and methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any). Readability is a goal of good code.

StateDataReader Overview

In this challenge you will be programming a series of code that is designed to read text files stored in the project folder by using the relative pathway. It has to be relative because your program will be loaded into a different machine, and if you use absolute pathfinding, there is a possibility the code may not work since it is almost certain the project URL will be different. The program will read 4 text files that have different file names. Each file has a list of students with these fields in the following format:

PREFIX FIRSTNAME LASTNAME

Not every student will have a prefix (i.e. Dr., Mr. Mrs.). There are no delimiters separating the fields for each student records in the file; all of the fields are separated by a space. Inside of each text files the student's record (their full name) will be listed one per line in no particular order. Not every text file contains the same number of students, and all have different students. The combined total of all four files will be 35 records.

After you are able to read the 4 text files, you will create three sorting algorithms. The first will sort the students in ascending alphabetical order (A to Z). Next you will sort the list of students in descending order (Z to A). Both of these algorithms can use the first letter of the record.

Next, you will be challenged with breaking the student name records apart and reorganizing them in ascending alphabetical order (A to Z) based upon their last name. The new order of the record should be the following:

LASTNAME, FIRSTNAME, PREFIX

Notice that there is now a comma between the Last Name and First Name. And if the record has a prefix, there should be a comma in between the First Name and Prefix. All of the records will be displayed in list boxes and will have a format that includes a number of the order of the record being printed. Examples are provided on the following page.

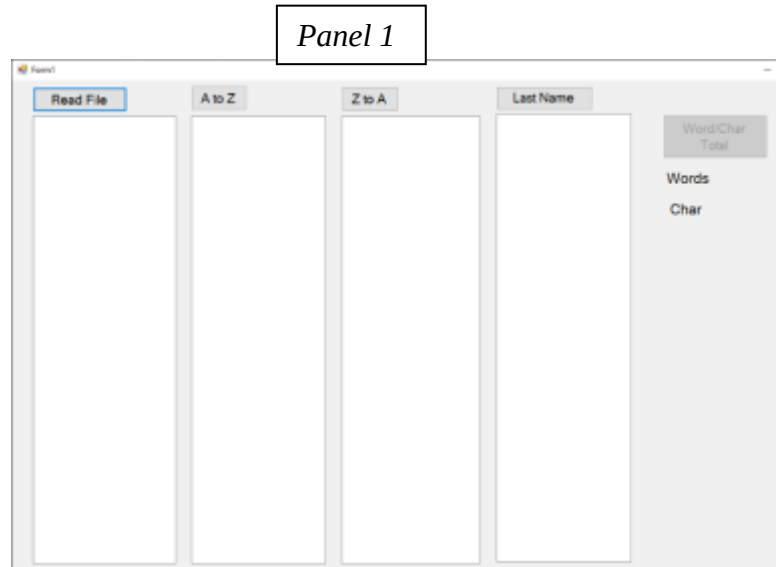
Finally, you must program the "Word/Char Total" buttons to count the total number of words from the files combined and the total number of characters (do not count symbols, numbers or spaces). You CANNOT hard code the values or this will result in loss of points. It must be calculated through the code's algorithm.

There are multiple ways to accomplish this challenge. However, you are limited to using the predefined form layout, and must use the predefined methods associated with the button clicking event. The program's success will be tied to your code being able to successfully read records of the text files. These records can be stored in a multitude of fashions. Choose the best one for your skill set. You **CANNOT** make an extra internal class. Your code design, the program, **MUST** run on any computer from the flash drive.

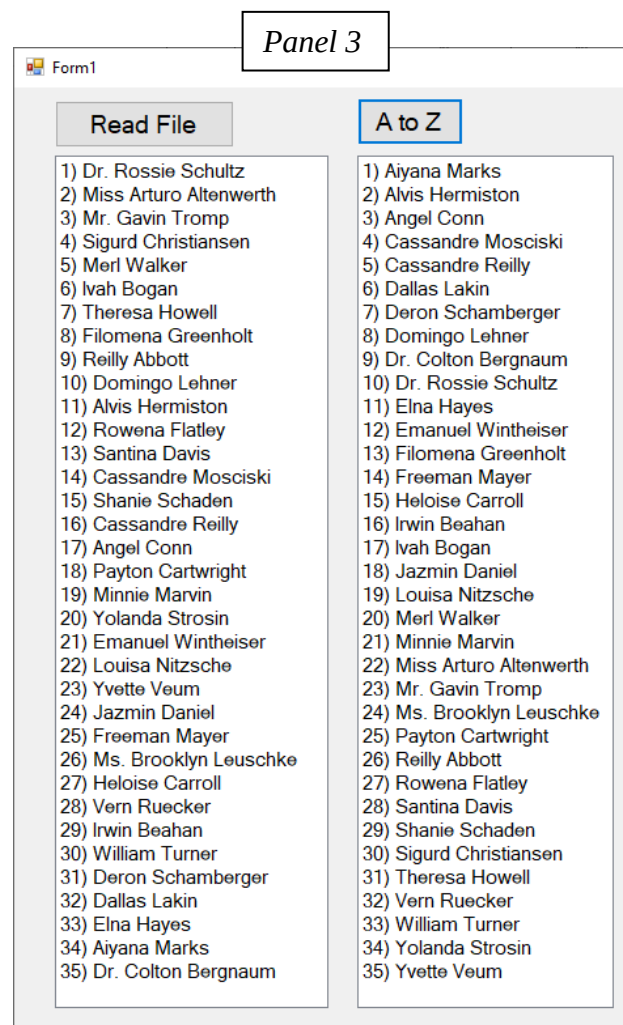
Input/Output

Panel 1 shows the form that has been provided for you will appear as such when the program runs initially. All of the sorting buttons are active; however, they will do nothing until the files have been read.

Panel 2 (below) demonstrates the four text files being read by the program when the “Read File” button is pressed. The order of the names in your list box may differ from these, however there should be 35 total names in the format shown below. And it should have the same names. Your results should match, excluding spacing. You will add the counting numbers and ‘)’.



Panel 3 (to the right) displays the output when the “A to Z” button has been pressed. All of the student records are alphabetized based upon the first letter in the records which could be the first name, or the prefix if one is present. This is intended to be a simple sorting algorithm demonstration. Your results should match, excluding spacing.



Panel 4 (on the right) is the inverse of **Panel 3** and shows the list of students being sorted in reverse alphabetical order. Again, this order is sorted based upon the first letter of the record. This is intended to be a simple sorting algorithm demonstration. Your results should match, excluding spacing.

Panel 5

Last Name

- 1) Abbott, Reilly
- 2) Arturo Altenwerth, Miss
- 3) Beahan, Irwin
- 4) Bogan, Ivah
- 5) Brooklyn Leuschke, Ms.
- 6) Carroll, Heloise
- 7) Cartwright, Payton
- 8) Christiansen, Sigurd
- 9) Colton Bergnaum, Dr.
- 10) Conn, Angel
- 11) Daniel, Jazmin
- 12) Davis, Santina
- 13) Flatley, Rowena
- 14) Gavin Tromp, Mr.
- 15) Greenholt, Filomena
- 16) Hayes, Elna
- 17) Hermiston, Alvis
- 18) Howell, Theresa
- 19) Lakin, Dallas
- 20) Lehner, Domingo
- 21) Marks, Aiyana
- 22) Marvin, Minnie
- 23) Mayer, Freeman
- 24) Mosciski, Cassandre
- 25) Nitzsche, Louisa
- 26) Reilly, Cassandre
- 27) Rossie Schultz, Dr.
- 28) Ruecker, Vern
- 29) Schaden, Shanie
- 30) Schamberger, Deron
- 31) Strosin, Yolanda
- 32) Turner, William
- 33) Veum, Yvette
- 34) Walker, Merl
- 35) Wintheiser, Emanuel

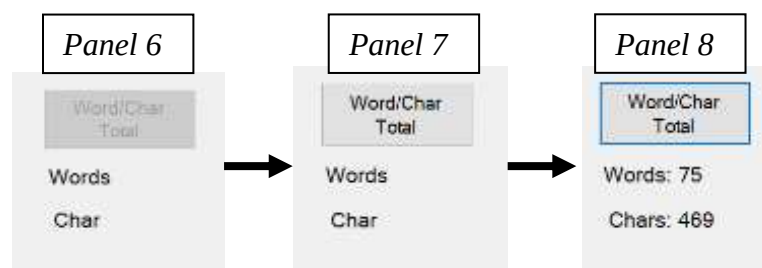
Panel 5 (on the left) will be more intensive than the other sorting tasks because you will need to reorder the student records in the following format. There are multiple ways to accomplish this goal. Your results should match, excluding spacing.

Panel 4

Z to A

- 35) Yvette Veum
- 34) Yolanda Strosin
- 33) William Turner
- 32) Vern Ruecker
- 31) Theresa Howell
- 30) Sigurd Christiansen
- 29) Shanie Schaden
- 28) Santina Davis
- 27) Rowena Flatley
- 26) Reilly Abbott
- 25) Payton Cartwright
- 24) Ms. Brooklyn Leuschke
- 23) Mr. Gavin Tromp
- 22) Miss Arturo Altenwerth
- 21) Minnie Marvin
- 20) Merl Walker
- 19) Louisa Nitzsche
- 18) Jazmin Daniel
- 17) Ivah Bogan
- 16) Irwin Beahan
- 15) Heloise Carroll
- 14) Freeman Mayer
- 13) Filomena Greenholt
- 12) Emanuel Wintheiser
- 11) Elna Hayes
- 10) Dr. Rossie Schultz
- 9) Dr. Colton Bergnaum
- 8) Domingo Lehner
- 7) Deron Schamberger
- 6) Dallas Lakin
- 5) Cassandre Reilly
- 4) Cassandre Mosciski
- 3) Angel Conn
- 2) Alvis Hermiston
- 1) Aiyana Marks

Panels 6, 7, & 8 (see below): the “Word/Char Total” button will need to be inactive (Panel 6) until the “Last Name” button is clicked and then it will become active (Panel 7). The button will need to be programmed to find the total number of words in the files and the total number of characters that exclude spaces and ‘.’ (periods), numbers, and ‘)’...only letters are counted. The totals are provided to help you determine if it was correctly analyzed.



Requirements

1. You must use the provided project called “StateDataReader”. This project will contain a “Form1.cs[design]” that has been already created in the organization and naming conventions.
2. The file “Form1.cs” has already been provided. This file contains all needed packages and the method headers have already been provided. You will program within this file.
3. No additional buttons or methods can be added to the form design.
4. Your contestant number must appear as a comment at the top of the main source code file.
5. User input (pressing the buttons) is needed for this program as the program will gain functionality through the importing and reading of four different text files.
6. You will be required to complete the given button listener methods that are already provided; you CANNOT create additional internal classes, the code can be completed without them.
7. The button that will be the catalyst to allow functionality to the other buttons is the [“Read File”] button. The rationale is that the files must be present in some type of data structure in order for the other buttons to be able to sort the values.
8. Records will be each individual line read by the program from the text files. Fields would be the categorization of the information in each record. For example, each record will contain the fields of prefix, first name and last name: Dr. Jose Patel.
9. The list boxes for the [“A to Z”] and [“Z to A”] buttons must sort based upon the first letter of the field in the record. In most cases this will be the first name; however, some records will have a prefix (Dr. Mrs. Mr.), in these instances use the first letter of the prefix since this is the leading character in the record. These sorting functions are intended to be easy.
10. The final sorting button (“Last Name”) will require the record to be manipulated based upon one of the following naming conventions:
 - a. LASTNAME, FIRSTNAME, PREFIX
 - b. LASTNAME, FIRSTNAME
11. The “Word/Char Total” button will generate data based upon the imported information only, not the concatenated numbers. There should be 75 words (this includes the prefixes) and 469 characters that do NOT include spaces or ‘.’ (periods).
12. Hard coding the data (i.e. directly typing the data) will eliminate any chance of earning points for that particular section and associated sections. All data must be generated from the file reading and from the manipulation/analysis of the strings.
13. The program will display the EXACT outputs as shown in the examples above. NOTE: there must be spacing between each field in the record and follow the naming convention. Extra spacing of 1-2 extra values will be ignored.
14. If a scorable item on the rubric says “Comments are required” you will only be able to earn points if that item has a comment with comment code provided in the rubric. The comment does not need to be a complete sentence, and only needs bring the graders attention to the main idea of what that method or segment of code performs. You do not need to list a comment for line by line explanation.
15. WARNING: all buttons have been named in advance and should not be change. They

will be prefixed with “btn”+Name. For example, **btnReadFile**. In addition, the buttons have an event associated with them, and those methods have already been created in advance. The naming convention for these button events which you will program are named. For example the method for the button **btnMax** has the method **btnReadFile_Click(object sender, EventArgs e) {...}** The **_Click(object sender, EventArgs e)** has been added for you. You will program these method based upon the requirements in the rubric (see next page). All buttons in this program will follow these guidelines, and you will program ALL of the buttons. A list of all buttons and methods is provided below.

- a. Button Name: **btnReadFile** Method Name: **btnReadFile_Click** Text: “Read File”
 - i. Prints to list box: **lstNames**
- b. Button Name: **btnAtoZ** Method Name: **btnAtoZ_Click** Text: “A to Z”
 - i. Prints to list box: **lstAtoZ**
- c. Button Name: **btnZtoA** Method Name: **btnZtoA_Click** Text: “Z to A”
 - i. Prints to list box: **lstZtoA**
- d. Button Name: **btnLastAtoZ** Method Name: **btnLastAtoZ_Click** Text: “Last Name”
 - i. Prints to list box: **lstLastNames**
- e. Button Name: **btnWordTotal** Method Name: **btnWordTotal_Click** Text: “Word/Char Total”
 - i. Word total prints to label: **lblWordTotal**
 - ii. Character total prints to label: **lblCharTotal**

Solution and Project		
The project is present on the flash drive		10 points
Program Execution		
The program runs from the USB flash drive: the form loads with no initial errors		15 points
<i>If the program does not execute, then the remaining items in this section receive a score of zero.</i>		
The program displays all buttons as active.		10 points
Before [“Read File”] button is clicked: Buttons [“A to Z”], [“Z to A”], and [“Last Name”] do NOT perform any actions when clicked. All or nothing.		10 points
The [“Read File”] button is clicked and the corresponding list box displays 35 records in random order and must match the format where the record is preceded with number and “)” for example: 1) Jose Montana.		20 points
The [“Read File”] button is clicked and the corresponding list box displays 35 records in random order and must match the format where the record is preceded with number and “)” for example: 1) Jose Montana.		20 points
The [“A to Z”] button is clicked and the corresponding list box displays 35 records in ascending order based upon the first character of the record and with the increasing numbering format 1 to 35.		20 points
The [“Z to A”] button is clicked and the corresponding list box displays 35 records in descending order based upon the first character of the record and with the decreasing numbering format 35 to 1.		20 points
The [“Last Name”] button is clicked and the corresponding list box displays 35 records in ascending order (1x) based upon the last name in the record; record has been reformatted to new naming convention (1x), commas are present (1x); count increases 1 to 35 (1x) < -10pts. per error >		40 points
The [“Word/Char Total”] button is not active until the [“Last Name”] button is pressed.		10 points
The [“Word/Char Total”] button is clicked and displays the total word count correctly (75 is the total for the provided files).		20 points
The [“Word/Char Total”] button is clicked and displays the total word character count correctly (469 is the total for the provided files).		20 points
Subtotal		/215 Points

Source Code Review		
A comment containing the contestant number is present		10 points
Form1.cs class: contains uses a single list to store the imported records from the text file. Comment is required; mark this comment as “SC1” at the beginning.		10 points
Form1.cs class: contains uses a separate single list to store the reorganized records into the new format (LASTNAME, FIRSTNAME, PREFIX). The code must have worked to get credit. Comment is required; mark this comment as “SC2” at the beginning.		10 points
Form1.cs class: <i>btnReadFile_Click</i> method uses static class Path to locate the files. The code must have worked to get credit. Comment is required; mark this comment as “SC3” at the beginning.		30 points
Form1.cs class: <i>btnReadFile_Click</i> method stores all of the file pathways into an array and requires no loops to get all file locations. The code must have worked to get credit. Comment is required; mark this comment as “SC4” at the beginning.		30 points
Form1.cs class: <i>btnReadFile_Click</i> method uses static class Path to locate the files. The code must have worked to get credit. Comment is required; mark this comment as “SC4” at the beginning.		10 points
Form1.cs class: <i>btnAtoZ_Click</i> method is commented explaining how the list has been sorted. The code must have worked to get credit. Mark this comment as “SC5” at the beginning.		10 points
Form1.cs class: <i>btnZtoA_Click</i> method is commented explaining how the list has been sorted in reverse order. The code must have worked to get credit. Mark this comment as “SC6” at the beginning.		10 points
Form1.cs class: <i>btnLastAtoZ_Click</i> method is commented explaining how the records have been broken into sections. The code must have worked to get credit. Mark this comment as “SC7” at the beginning.		30 points
Form1.cs class: <i>btnLastAtoZ_Click</i> method is commented explaining how the records have been reorganized into the TWO different types of naming conventions. The code must have worked to get credit. Mark this comment as “SC8” at the beginning.		30 points
Form1.cs class: <i>btnWordTotal_Click</i> method is commented explaining the total words were counted. The code must have worked to get credit. Mark this comment as “SC9” at the beginning.		20 points
Form1.cs class: <i>btnWordTotal_Click</i> method is commented explaining the total characters were counted. The code must have worked to get credit. Mark this comment as “SC10” at the beginning.		20 points
Subtotal		/220 Points
Total Points		/435 Points